State-Space Search in Prolog and Tournament Scheduling in ASP

Exam project — Artificial Intelligence and Laboratory
Logic Programming and Answer Set Programming module

31 October 2026

Abstract. The project implements the two required activities. The first is a domain-independent A* and IDA* engine in SWI-Prolog, applied to a multi-exit labyrinth and to the 8-puzzle. The engine exposes a small problem-descriptor contract, keeps the domains fully separate, and reports comparable statistics. The second is a Clingo encoding that schedules one round of a Pokémon League tournament with dynamic arrivals, a single BYE on odd fields, no repeated pairings, and no repeated BYEs. The official model optimises score-balanced pairings lexicographically: unequal-score matches first, then total absolute difference, then the BYE's score. A sum-only baseline is included to show that the two objectives are not equivalent. The implementation is validated by 46 Prolog tests and 21 Python tests, including independent breadth-first oracles for both search domains.

1 Introduction

The project covers two ways of encoding search problems. Activity 1 treats search as an algorithmic process over explicit states. Activity 2 treats a combinatorial decision problem declaratively, letting Clingo generate, test, and optimise answer sets. Both activities share the same requirement: the solution must be correct, reproducible, and accompanied by an honest experimental comparison.

All code, comments, tests, this report, and the slides are in English. The repository contains frozen benchmark data, so every number below can be regenerated with the commands in Appendix A.

2 Activity 1: state-space search in Prolog

2.1 Contract and architecture

The engines are independent of the domains. A problem is a term `problem(DomainModule, Config)`, and each domain exports six predicates:

Predicate	Meaning
`initial_state/2`	the initial state
`goal_test/2`	true for any goal state; the labyrinth has several exits
`successors/3`	a list of `move(Action, Cost, NextState)`
`heuristic/3`	an admissible, non-negative estimate
`state_key/2`	a ground canonical key for the visited maps
`state_label/2`	a human-readable label for output

The public engine API is `a_star(Problem, Options, Result, Stats)` and `ida_star(Problem, Options, Result, Stats)`. `Result` is `solved(Path, Cost)`, `no_path`, or `limit_exceeded`; `Path` is a list of `step(Action, Cost, State)` terms, with the initial state implicit. Options are `heuristic(Name)`, `max_expansions(N)`, and `timeout(Seconds)`. Both engines return the same statistics dictionary, which makes the benchmark comparison meaningful.

The separation is enforced by the directory structure: `prolog/search/` never imports a domain. Domains may optionally export `with_heuristic/3` for heuristic selection and `solvable/1` for a fast unsolvable precheck. The engine uses those hooks only when the predicates exist, so the contract stays small.

2.2 A*

A* keeps a frontier ordered by $f(n) = g(n) + h(n)$. The implementation uses SWI-Prolog's `library(heaps)` with the key `f(F, -G, Seq)`. The first component orders by f; the negative g makes ties prefer the deeper node; the monotonic sequence number removes the remaining nondeterminism. The negative g is also the reason the heap key is built as a compound term rather than a bare number: heap comparison follows standard order on the whole term.

`library(heaps)` offers no decrease-key operation. Instead, an improved successor is pushed again and the old entry is discarded lazily when it is popped: an entry is stale when its stored g is worse than the best known g for the same state. Best scores and parent pointers are stored in `library(assoc)` trees keyed by the domain's canonical state key. If a cheaper path reaches an already expanded state, the state is reopened. This best-g policy is what preserves optimality when the heuristic is admissible but inconsistent.

The goal test is performed when a node is popped, not when it is generated. Testing on generation would return a suboptimal goal when an inconsistent heuristic makes a goal visible before all cheaper paths have been explored. Expansions count non-stale pops; generated nodes count every successor produced; stale pops and reopenings are counted separately.

2.3 IDA*

IDA* replaces the frontier with a sequence of depth-first searches. The first threshold is $h(\text{start})$; after a search that exceeds the bound, the next threshold is the smallest f value that was cut off. The search stops when a goal is popped inside the bound or when a complete iteration cuts nothing, which proves that no solution exists under the current threshold.

Cycle checking uses only the current path. This is the textbook memory-bound behaviour: space is $O(d)$, but a state can be expanded again through a different branch. Every visit counts as an expansion, including re-expansions across iterations; maximum depth and stack size are reported separately. The unsolvable precheck is shared with A*: the 8-puzzle rejects odd inversion parity before any search, and the labyrinth rejects maps whose exits are all unreachable.

2.4 Domains and heuristics

Labyrinth. A map is a module exporting `cols/1`, `rows/1`, `start/1`, `exit/1`, and `blocked/1`. States are `pos(Row, Column)`, 1-based; moves are the four orthogonal neighbours with unit cost. The Manhattan and Euclidean heuristics take the minimum distance over every exit. The minimum of 1-Lipschitz functions is 1-Lipschitz, so both heuristics are consistent as well as admissible. An unreachable exit still has a finite geometric distance; it can only lower the estimate, never make it inadmissible. The multi-exit map contains such a sealed exit on purpose.

8-puzzle. A state is a flat nine-element list in row-major order with `0` as the blank. Moves swap the blank with an orthogonal tile. Solvability is the parity of the inversion count of the tiles with the blank removed. The misplaced-tiles and Manhattan heuristics ignore the blank; both are admissible and consistent, and Manhattan dominates misplaced tiles. Twelve solvable instances cover optimal costs from 2 to 16.

2.5 Correctness and tests

The Prolog suite contains 46 tests:

- 11 engine tests on two hand-built graphs, including a graph that forces reopening under an inconsistent heuristic and a graph that forces lazy deletion of a stale frontier entry;
- 18 labyrinth tests, with 14 generated sub-tests, covering map invariants, move legality, admissibility and consistency on every free cell, reachability of the sealed exit, and optimal-cost agreement with an independent breadth-first oracle;

- 17 8-puzzle tests covering parity, move legality, heuristic values, admissibility on samples, and optimal-cost agreement with a breadth-first oracle.

The oracles matter: they do not use the engines or the heuristics, so an error shared by A* and IDA* would still be detected. The engine tests additionally check `max_expansions` and `timeout(0)`, and replay every returned path through the domain contract.

2.6 Benchmark methodology

The benchmark matrix is 3 labyrinth maps × {Manhattan, Euclidean} × {A*, IDA*} and 12 puzzle instances × {misplaced, Manhattan} × {A*, IDA*}: 60 configurations, five repetitions each, 300 runs. A garbage collection runs before every repetition. The timeout is 60 seconds. The harness writes one row per repetition (`runs.csv`) and a median row per configuration (`summary.csv`). The frozen run took about 14 seconds on the development machine; CPU times below are medians and should be read as relative, not as absolute, measurements.

Peak RSS is left empty in the CSV. SWI-Prolog's `statistics/2` reports current memory, not peak memory, and mixing the two would be misleading; peak RSS must be measured in a fresh process. Frontier size and maximum depth are reported as the memory-shape indicators instead.

2.7 Results

Map	Solver	Heuristic	Cost	Expanded	CPU (ms)
default (10×10)	A*	Manhattan	16	44	4.3
	A*	Euclidean	16	41	2.9
	IDA*	Manhattan	16	3,465	77.0
	IDA*	Euclidean	16	73,471	1,802.8
multi-exit (9×9)	A*	Manhattan	12	35	2.8
	A*	Euclidean	12	40	2.9
	IDA*	Manhattan	12	1,100	21.9
	IDA*	Euclidean	12	15,611	308.8
open (8×8)	A*	Manhattan	7	8	1.3
	A*	Euclidean	7	25	2.2
	IDA*	Manhattan	7	8	1.0
	IDA*	Euclidean	7	609	10.2

Table 1: Labyrinth results, medians of five repetitions. All solvers find the same optimal cost on every map.

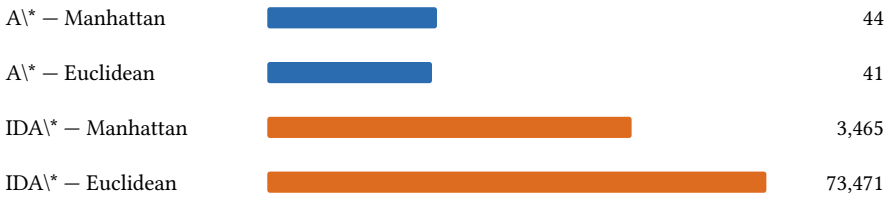


Figure 1: Expanded nodes on the default labyrinth. The bar length is logarithmic in the node count; the labels give the actual values.

On the default map, A* expands 44 nodes with Manhattan and 41 with Euclidean. The difference is small and, in this run, Euclidean even wins slightly. This is not a contradiction: Manhattan dominates Euclidean pointwise, but a dominating consistent heuristic does not by itself guarantee fewer expansions under a particular tie-breaking rule. With unit costs and a 10×10 grid, both estimates order most states similarly, and the heap tie-break can decide the rest.

The IDA* comparison is much sharper. Manhattan needs 3,465 expansions and 77 ms; Euclidean needs 73,471 expansions and 1.8 s. IDA* is sensitive to the number of threshold iterations, and a weaker estimate adds whole iterations. The same pattern repeats on the multi-exit map (1,100 versus 15,611) and, on a smaller scale, on the open map (8 versus 609). The cost is identical in every case, which is the expected optimality result.

Instance	Solver	Heuristic	Cost	Expanded	CPU (ms)
easy_8	A*	Manhattan	8	9	1.1
	IDA*	Manhattan	8	14	0.5
	A*	misplaced	8	9	0.7
	IDA*	misplaced	8	14	0.4
medium_12	A*	Manhattan	12	22	1.7
	IDA*	Manhattan	12	87	2.9
	A*	misplaced	12	64	4.2
	IDA*	misplaced	12	326	7.8
medium_16	A*	Manhattan	16	48	3.9
	IDA*	Manhattan	16	180	6.0
	A*	misplaced	16	341	24.1
	IDA*	misplaced	16	2,419	60.9

Table 2: Representative 8-puzzle results. Manhattan's advantage grows with depth: on `medium_16` it reduces A* expansions from 341 to 48 and IDA* expansions from 2,419 to 180.

Across all twelve puzzle instances, A* with Manhattan expands 323 nodes in total, A* with misplaced tiles 1,577, IDA* with Manhattan 799, and IDA* with misplaced tiles 7,550. The ordering is the same as in the labyrinth: the stronger heuristic wins, and IDA* pays for re-expansion. The puzzle depths are small enough that the IDA* memory advantage is not visible in CPU time here; the labyrinth makes the trade-off much clearer.

2.8 Discussion

The experiments support three conclusions. First, A* and IDA* return the same optimal costs on every instance, and the independent oracles confirm those costs. Second, the heuristic matters much more for IDA* than for A* in these domains, because a weaker estimate multiplies threshold iterations. Third, IDA* uses linear memory but can expand orders of magnitude more nodes: on the default labyrinth with Euclidean it expands about 1,670 times as many nodes as A*.

A*'s larger memory footprint is visible in `max_frontier`, not in the CSV's empty RSS column. The parent map and the best-g map also grow with the number of distinct states. An honest comparison therefore reports both the frontier size and the expansion counts, and does not claim that one algorithm is universally better.

3 Activity 2: ASP tournament scheduling

3.1 Problem

A Pokémon League tournament has n active players with current standings measured in wins. A round assigns every player at most one opponent. On an odd field exactly one player receives the BYE, which counts as a win; on an even field every player is paired. A pair may meet at most once in the whole tournament, a player may receive the BYE at most once, and no player withdraws. New players can join between rounds. The input therefore carries the full cumulative history, not only the previous round.

A single round is a matching problem and is polynomial; ASP is used here because the constraints are declarative and because the optimisation is part of the language. The model makes no NP-hardness claim for one round.

3.2 Encoding

The shared file `tournament_common.lp` implements generate–test:

```
candidate(X, Y) :- player(X), player(Y), X < Y,
                   not played(X, Y), not played(Y, X).
{match(X, Y) : candidate(X, Y)}.

incident(P, Q) :- match(P, Q).
incident(P, Q) :- match(Q, P).
```

```
:- player(P), 2 { incident(P, Q) : player(Q) }.

bye(P) :- player(P), not has_match(P).
```

The guard `X < Y` generates each unordered pair once, so `match(a,b)` and `match(b,a)` cannot become two independent decisions. The cardinality constraint counts both orientations through `incident/2`. `bye/1` is derived from "no incident match"; the constraints then require exactly one BYE on an odd field and none on an even field. This also guarantees that every non-BYE player is paired.

The remaining integrity constraints reject a repeated pair, a repeated BYE, a BYE player with a match, and malformed input (no players, a new player not in `player/1`, a missing or conflicting score). `had_bye/1` is history; `bye/1` is the newly assigned output. The two are never mixed.

3.3 Objectives

The official model `tournament.lp` uses three weak-constraint levels:

```
#minimize{1@3, X, Y : unequal_pair(X, Y)}.
#minimize{D@2, X, Y : diff(X, Y, D)}.
#minimize{S@1, P : bye_score(P, S)}.
```

Clingo optimises the highest priority first, so level 3 minimises the number of matches between players with different scores, level 2 minimises the total absolute difference, and level 1 breaks remaining ties in favour of giving the BYE to the lowest-standing player. The absolute difference is computed by two guarded rules (`SX >= SY` and `SX < SY`), never by a signed subtraction, and each tuple carries `X, Y` because equal cost values would otherwise collapse into one element.

The baseline `tournament_sum.lp` keeps the same hard constraints and uses only `#minimize{D@1, X, Y : diff(X, Y, D)}`. The two objectives are not equivalent. With players scored $0, 0, 1, 1, 2, 3$ where the two zero-score players have already played each other, the official model keeps the unique equal-score pair `c–d` and accepts a total difference of 5; the sum model breaks that pair and reaches a total difference of 3.

Model	Pairings	Equal pairs	Total difference
official	`a–e, b–f, c–d`	1	5
sum	`a–c, b–d, e–f`	0	3

Table 3: Counterexample with scores $0, 0, 1, 1, 2, 3$ and `played(a,b)`. The official model maximises equal-score pairings first; the sum model trades the equal pair for a smaller total gap.

3.4 Driver, validation, and multi-round harness

`asp/driver.py` loads an instance, solves it with the selected model, and validates the result independently of the encoding: canonical pairs, active players, no repeated pair or BYE, one opponent per player, the correct number of BYEs, and full coverage of the field. It also offers a minimal multi-round harness. With `--advance PATH` it applies the winners, adds `--win-points` for each win and for the BYE, appends `played/2` and `had_bye/1`, and writes the cumulative instance for the next round. When no explicit results are supplied, the lexicographically smaller identifier wins; this is a documented deterministic default, not a claim about real match outcomes.

For an infeasible instance the driver returns exit code 2 and runs a small relaxation ladder. It first tries to allow a repeated BYE; if that is still infeasible it allows repeated pairings and reports which relaxation was needed. The diagnostic is deliberately simple and exists to explain UNSAT, not to produce a published schedule.

3.5 Results

Round	Players	Schedule	BYE	Cost
1	5	`alice–bob, carol–dave`	erin	`1,1,0`
2	6	`alice–frank, bob–carol, dave–erin`	none	`1,4,0`
3	7	`bob–dave, carol–erin, frank–grace`	alice	`2,4,5`

Table 4: Official model on the three progressive instances. Round 3 has a prior BYE and six prior matches; the first two objective levels force the Frank–Grace equal-score pair and, with it, Alice as the BYE player.

The three instances exercise the intended behaviour. Round 1 is an odd field with no history: two pairs and one BYE. Round 2 adds a player and becomes an even field: three pairs and no BYE. Round 3 is odd again, with a prior BYE and a dense history. The first objective level fixes the Frank–Grace equal-score pair; the second level then makes Alice the only BYE choice that keeps the total difference at its minimum, because Erin already had a BYE and the remaining pairings are constrained by the played history. The third level has no remaining tie to break in this instance.

The tests cover these invariants on the committed instances and on synthetic ones, the official-versus-sum contrast, the arrival of a new player, the infeasible instance, and the score/history update of the multi-round harness. The 21 Python tests also include the cyclomatic-complexity check.

4 Limitations and threats to validity

Benchmark scope. The puzzle instances are easy and medium (optimal cost at most 16), which keeps IDA* meaningful instead of turning the benchmark into a timeout study. The results should not be extrapolated to random 8-puzzles, where IDA* with Manhattan is known to be much more expensive.

Timing. CPU times come from `statistics(cputime)` on one development machine. They are medians of five runs and are useful for relative comparisons only. Peak RSS is not reported in-process, for the reason given in Section 2.6.

Heuristic quality. Only the required heuristics are implemented. Linear conflict, pattern databases, and an exact BFS distance transform would change the expansion counts, but they were deliberately left out of scope.

ASP objective. The lexicographic model encodes one reasonable reading of "maximise equal-score pairings". The sum baseline shows that the choice is substantive. The BYE fairness level is necessarily a tie-break after the first two levels; it does not override them. Real tournaments may also need colour balance, repeat-window constraints, or explicit results, which are outside the project scope.

Scoring. `score/2` stores wins, and a BYE adds one, following the project statement. If a different points table is required, `--win-points` changes the award without touching the ASP encoding, because the model only compares current scores.

5 Conclusion

The project delivers both activities as a tested, reproducible system. The Prolog side separates a generic A*/IDA* engine from two domains, preserves optimality with inconsistent heuristics through reopenings, rejects unsolvable instances early, and validates its costs against independent breadth-first oracles. The ASP side encodes the tournament rules directly, optimises equal-score pairings lexicographically, and demonstrates with a concrete counterexample why a plain sum objective is not equivalent. The benchmark and the report are generated from committed data, and the same commands run in CI.

6 References

- P. E. Hart, N. J. Nilsson, and B. Raphael. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." **IEEE Transactions on Systems Science and Cybernetics**, 4(2):100–107, 1968.

- R. E. Korf. “Depth-First Iterative-Deepening: An Optimal Admissible Tree Search.” **Artificial Intelligence**, 27(1):97–109, 1985.
- S. Russell and P. Norvig. **Artificial Intelligence: A Modern Approach**. 4th edition, Pearson, 2020.
- M. Gebser, R. Kaminski, B. Kaufmann, and T. Schaub. **Answer Set Solving in Practice**. Morgan & Claypool, 2012.
- Potassco. **Clingo 5.8 User Guide** and **Potassco Guide**. potassco.org/clingo.
- SWI-Prolog. **Manual: library(heaps), library(assoc), plunit**. swi-prolog.org.

7 Appendix A: reproducibility

```
# Python environment (Python 3.11+)
python -m pip install -r requirements-dev.txt

# Prolog tests
swipl -g run_tests -t halt tests/test_search.pl
swipl -g run_tests -t halt tests/test_labyrinth.pl
swipl -g run_tests -t halt tests/test_puzzle8.pl

# Python and ASP tests
pytest -v tests/

# Benchmark (60 configurations × 5 repetitions)
swipl -g "bench:main" -t halt prolog/benchmarks/bench.pl

# ASP round and next-round instance
python asp/driver.py --instance asp/instances/round1.lp
python asp/driver.py --instance asp/instances/round1.lp \
    --advance /tmp/round2.lp --win-points 1

# Report and slides
typst compile report/report.typ
typst compile slides/slides.typ
```